

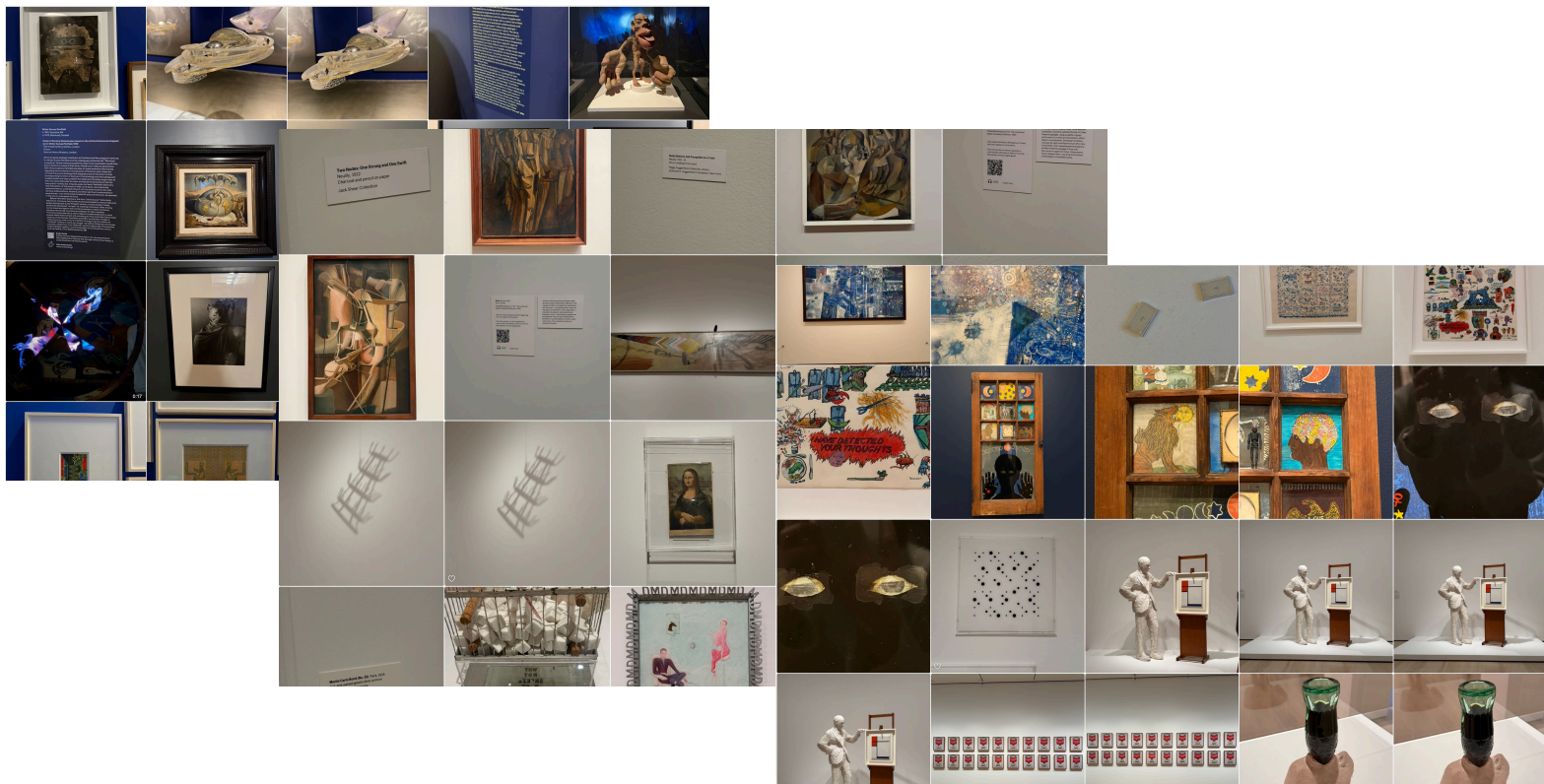
Intro to Web Development

Personal Art Collection Website

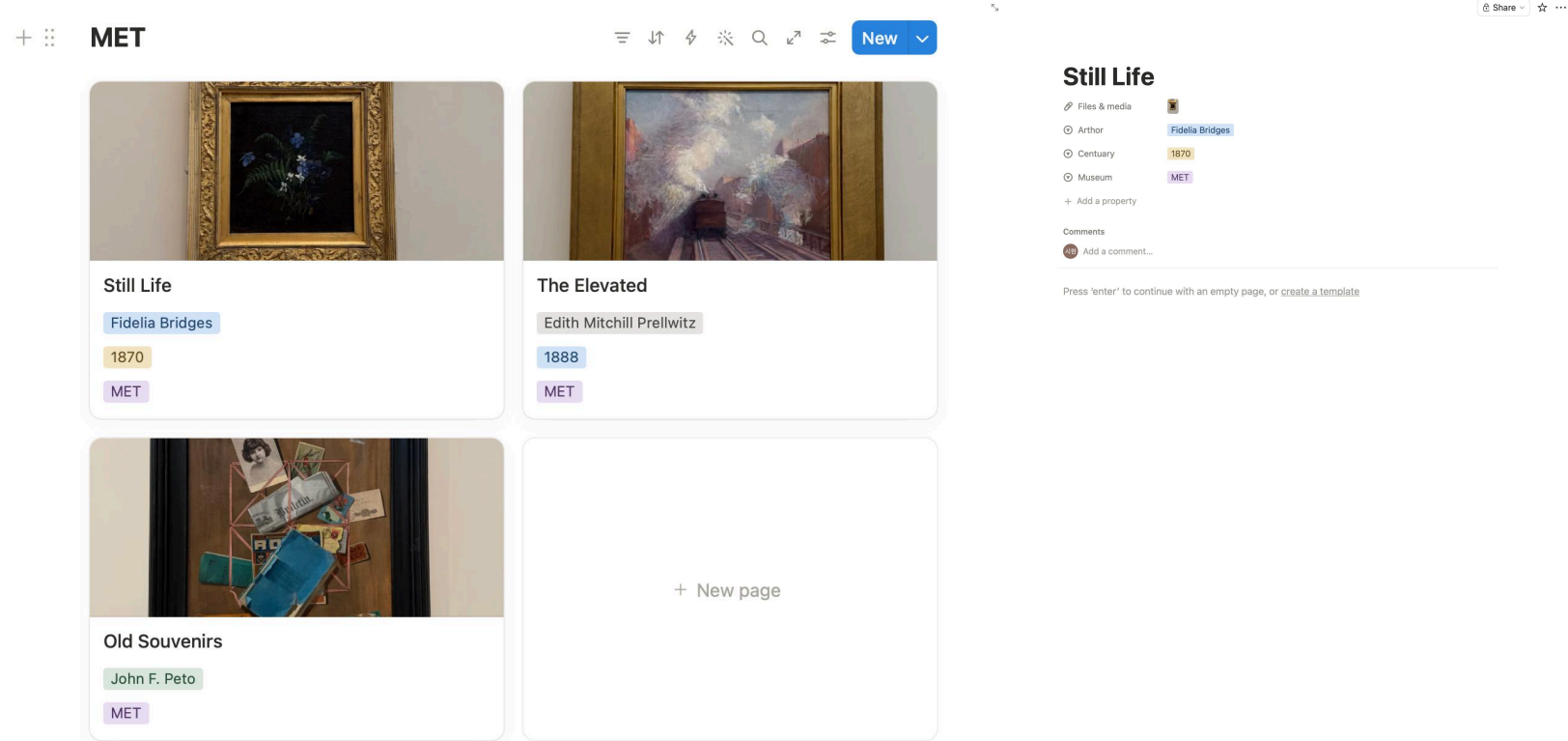
A personal digital gallery for art encountered during museum visits

Problem

My album after a visit to the museum

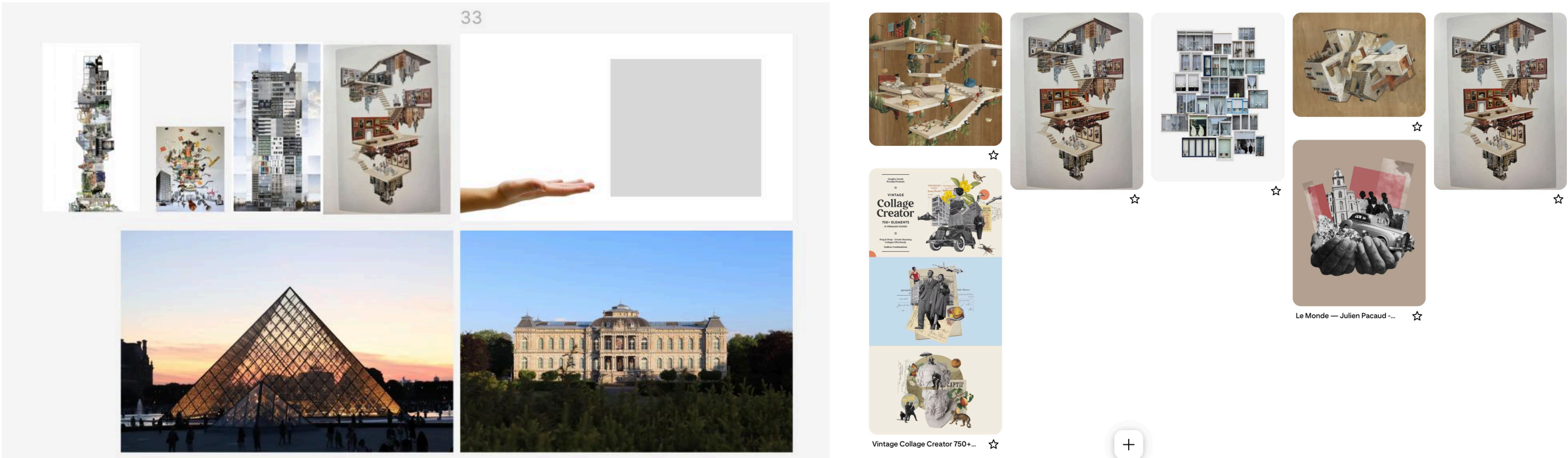
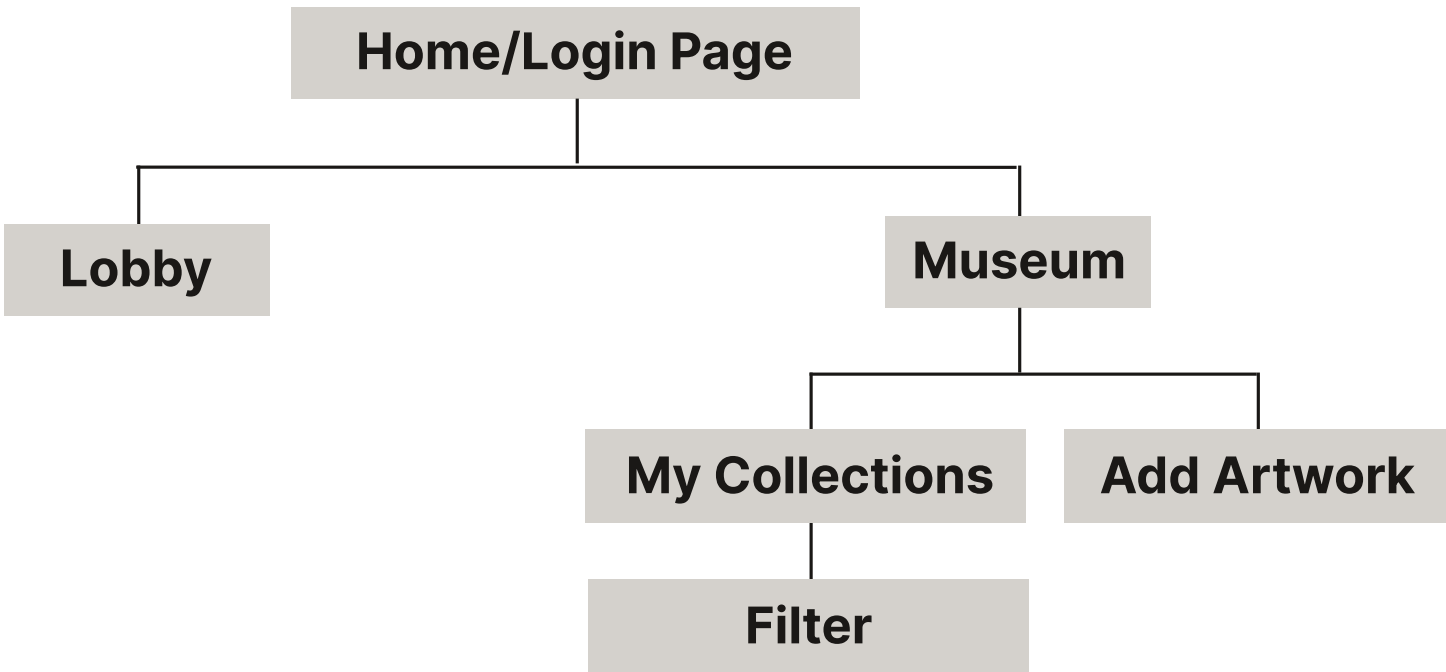


Tried doing it with Notion, but it didn't work out well



Solution

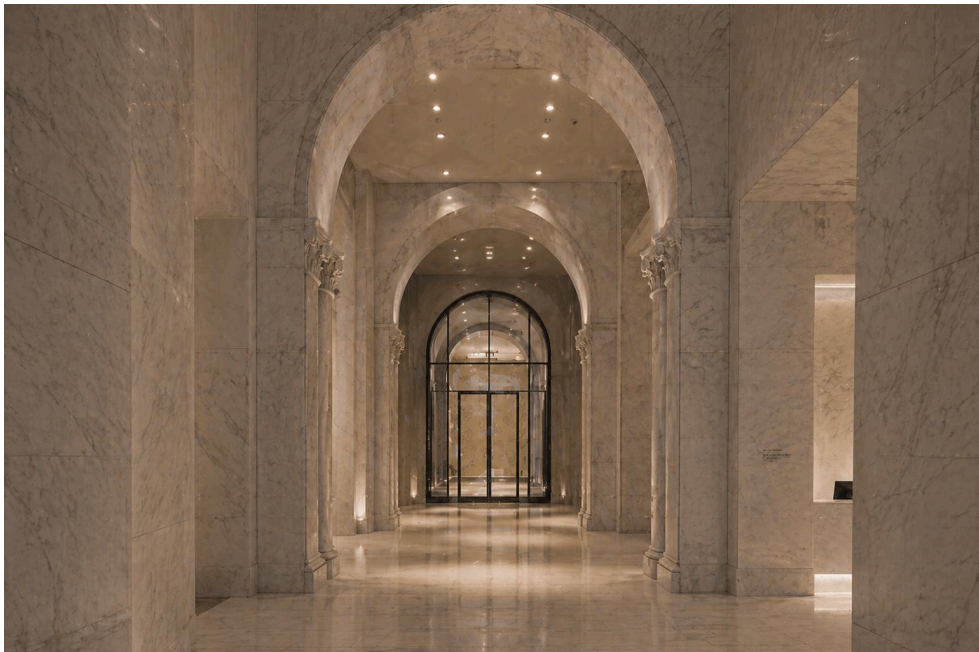
IA & Design Reference



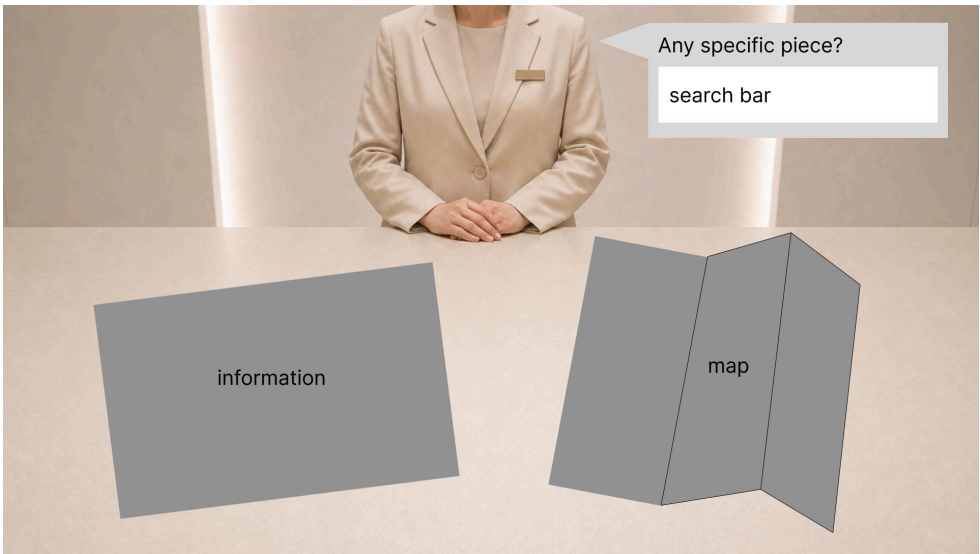
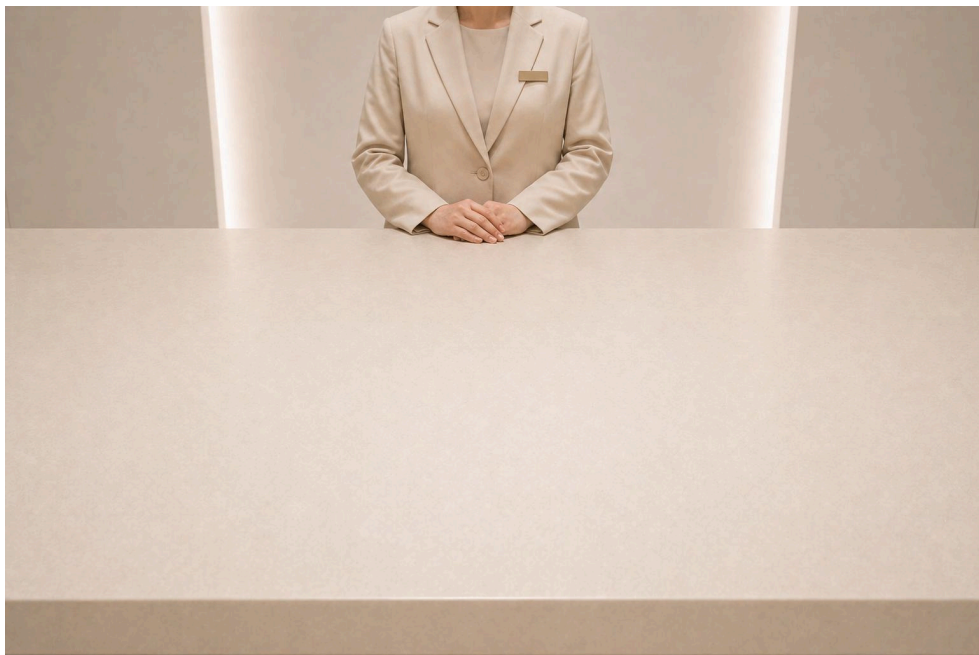
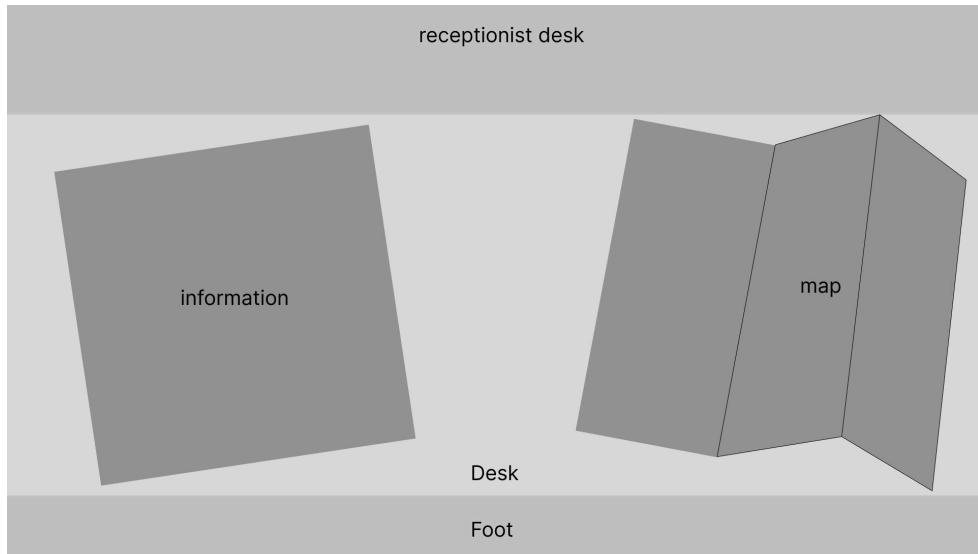
Design Process

Background Images

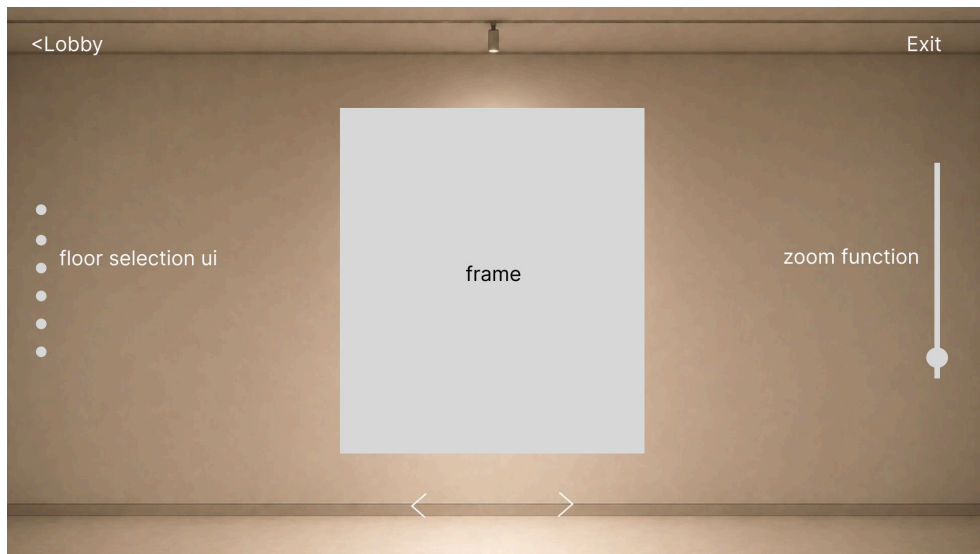
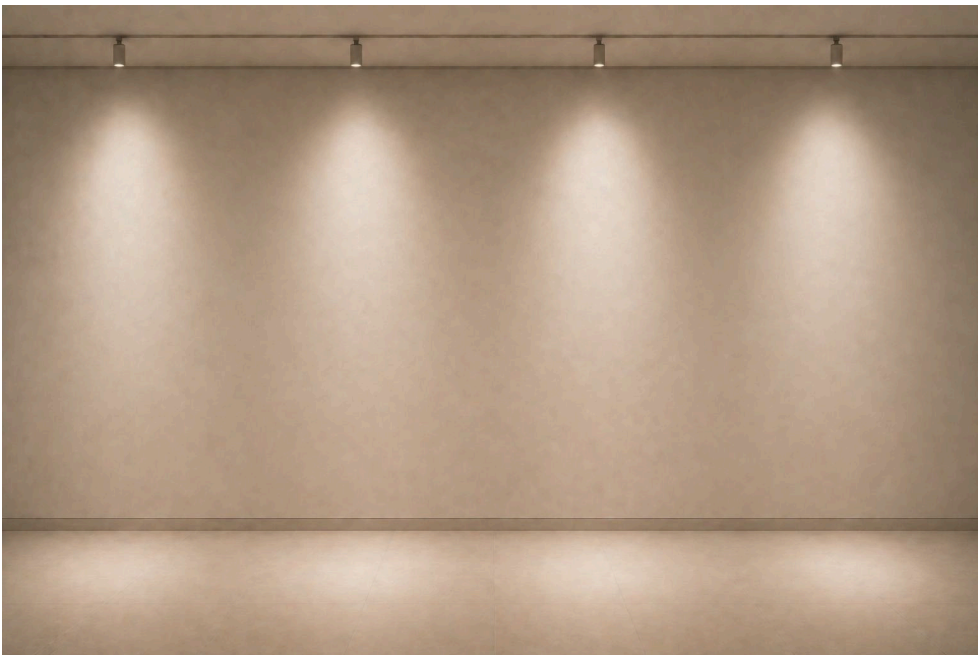
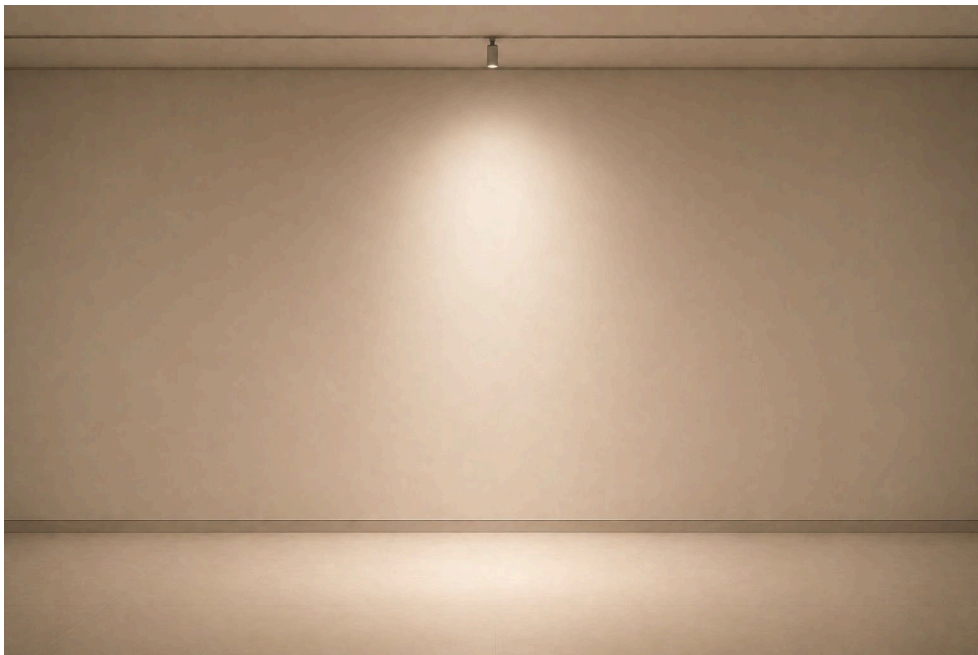
Entrance



Lobby

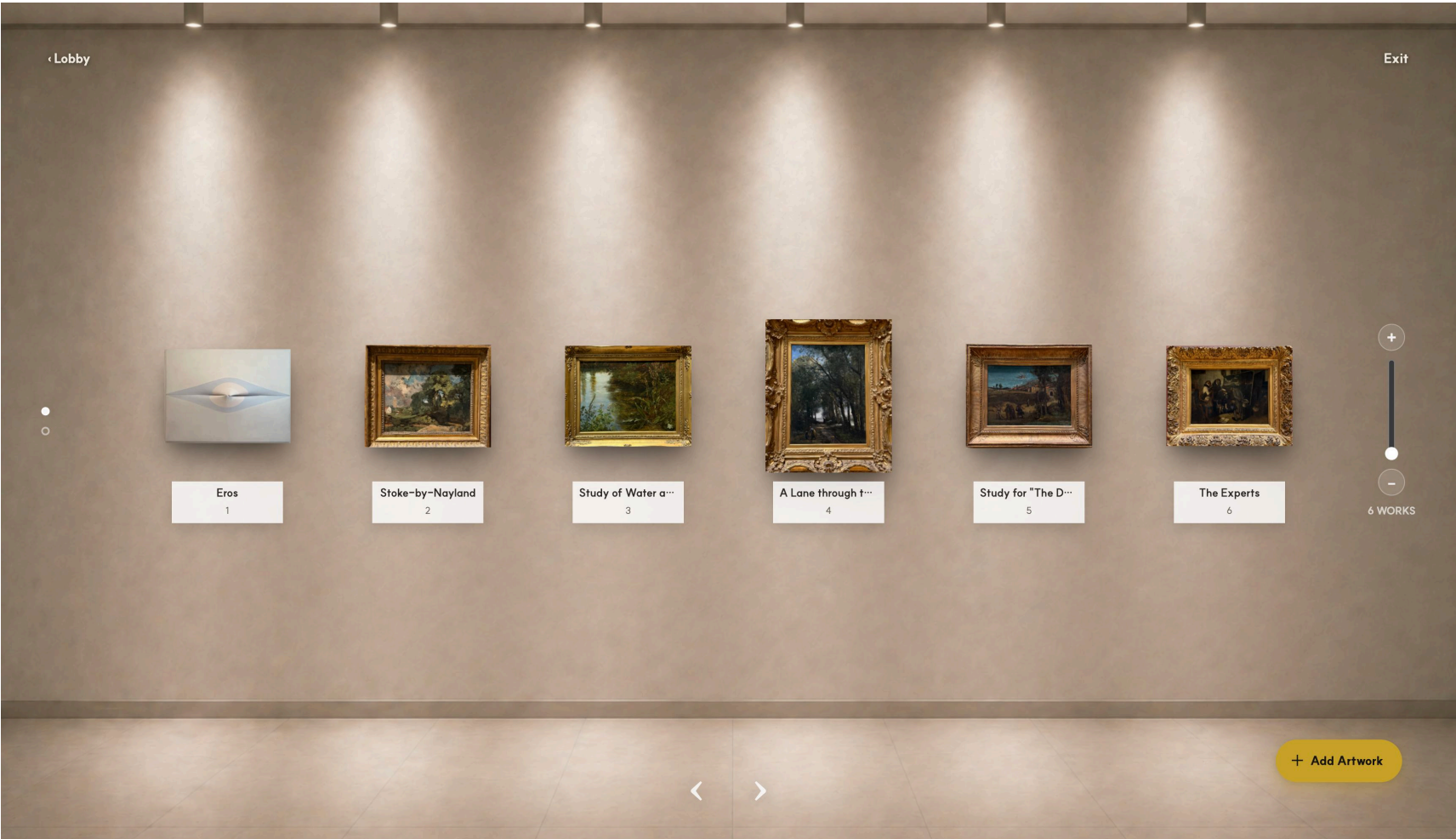
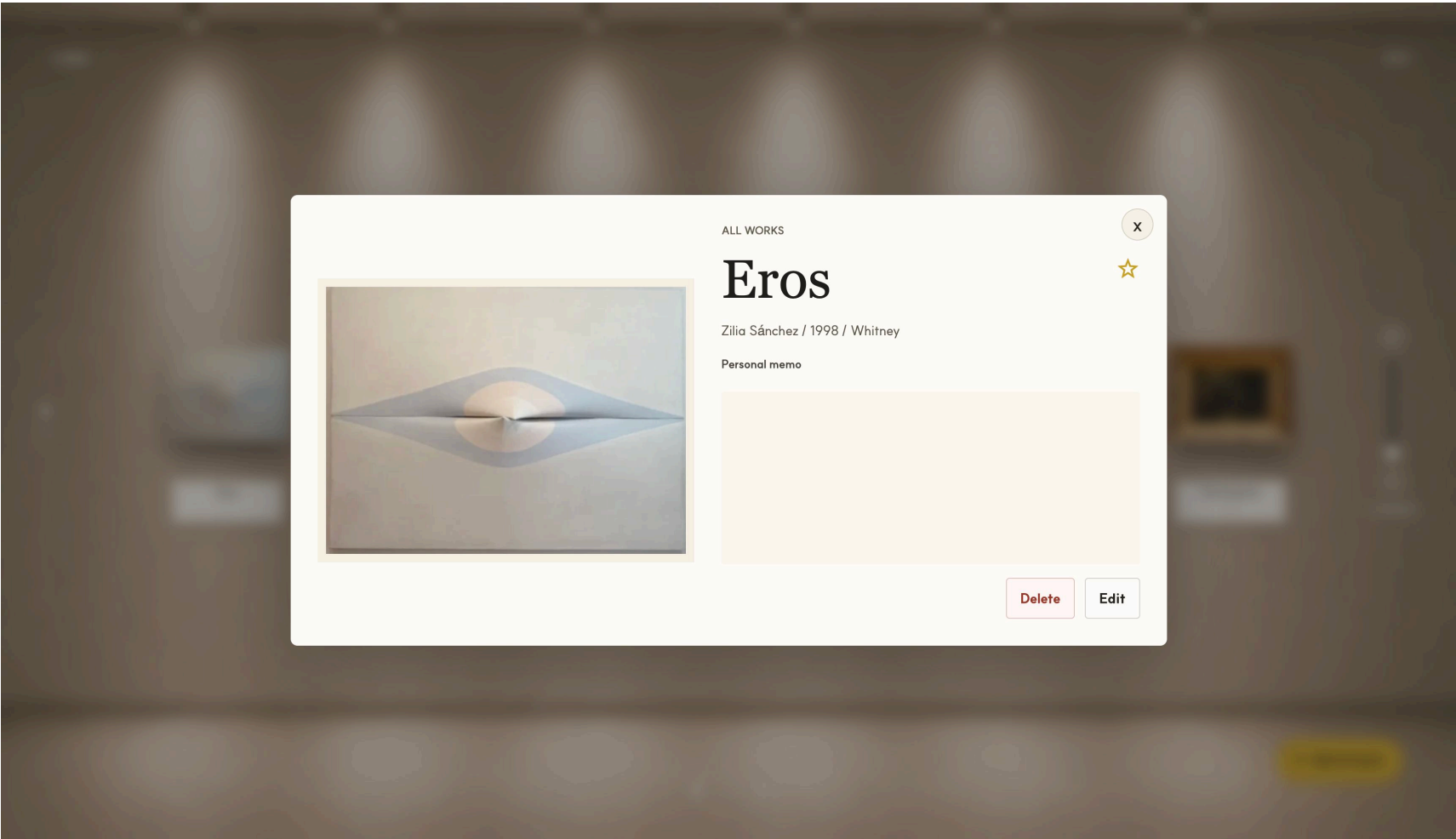
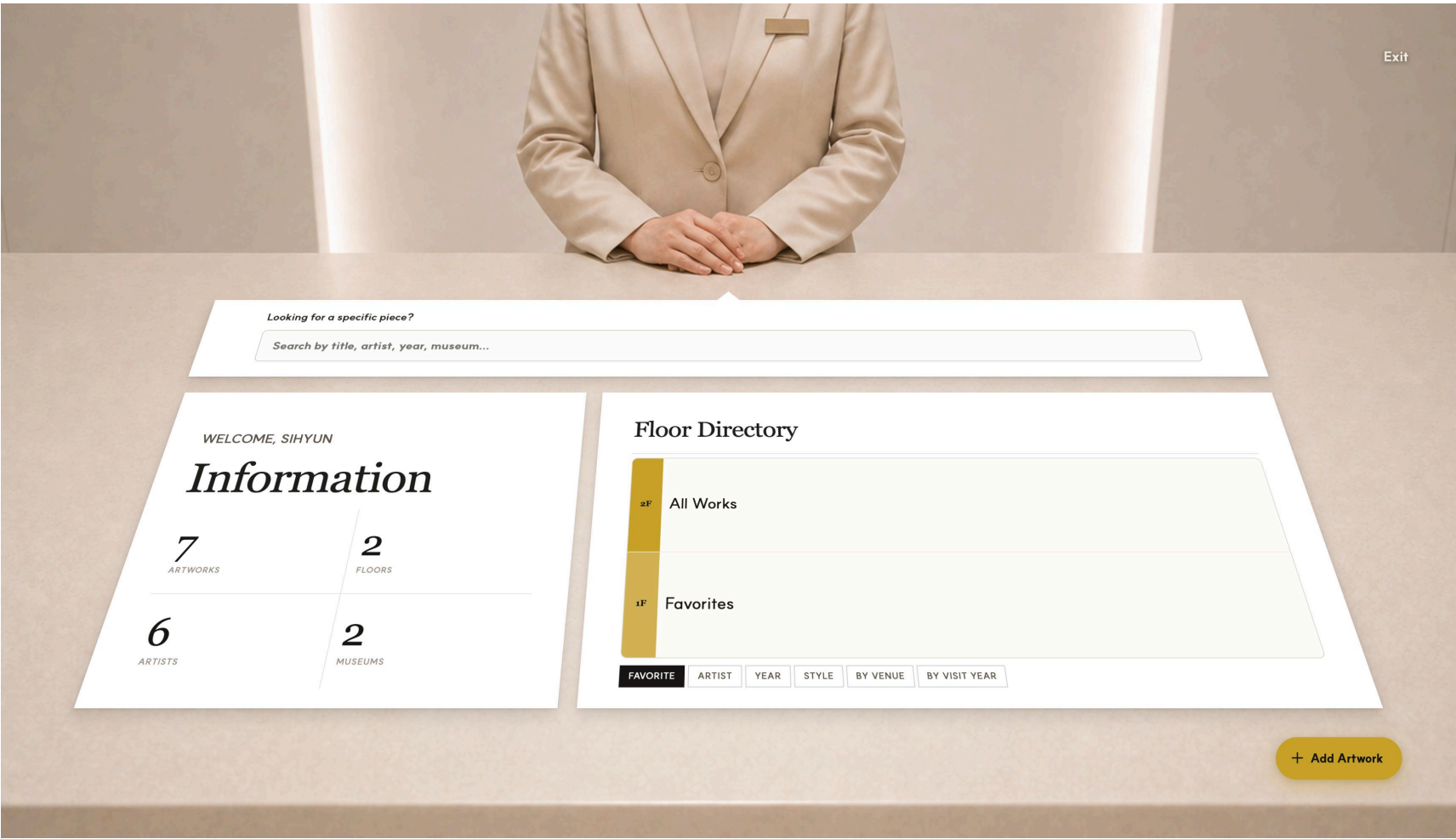
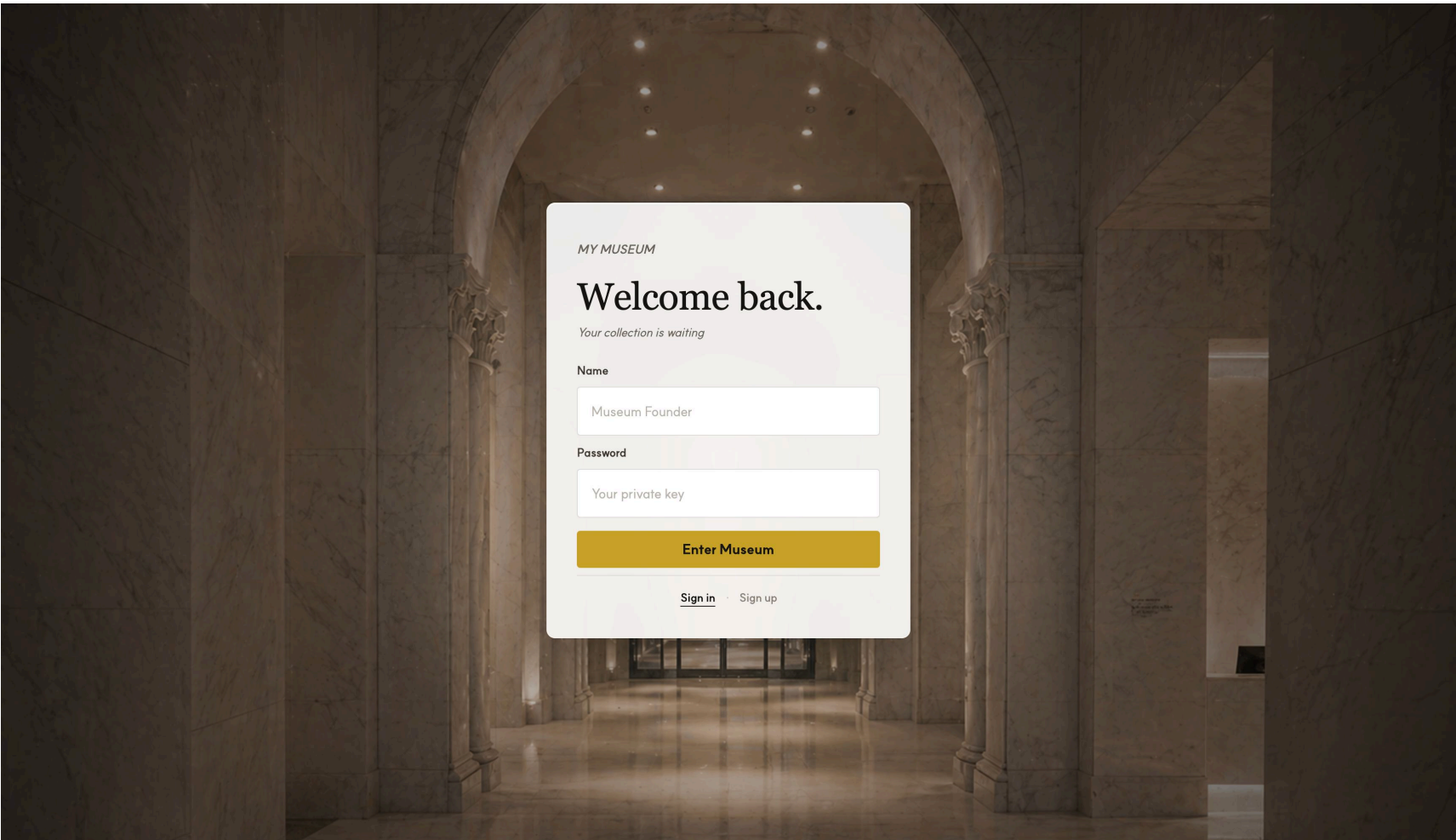


Inside



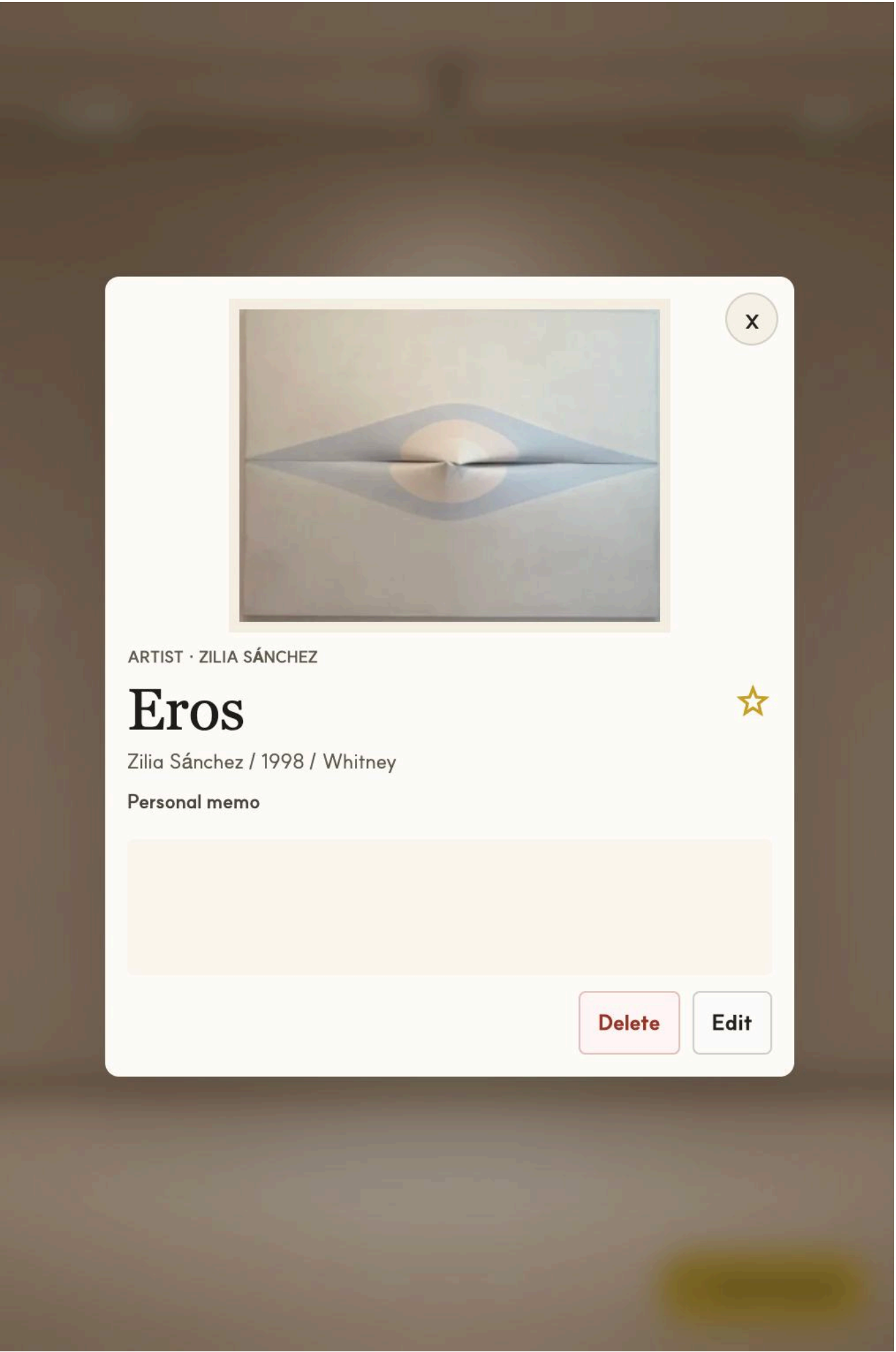
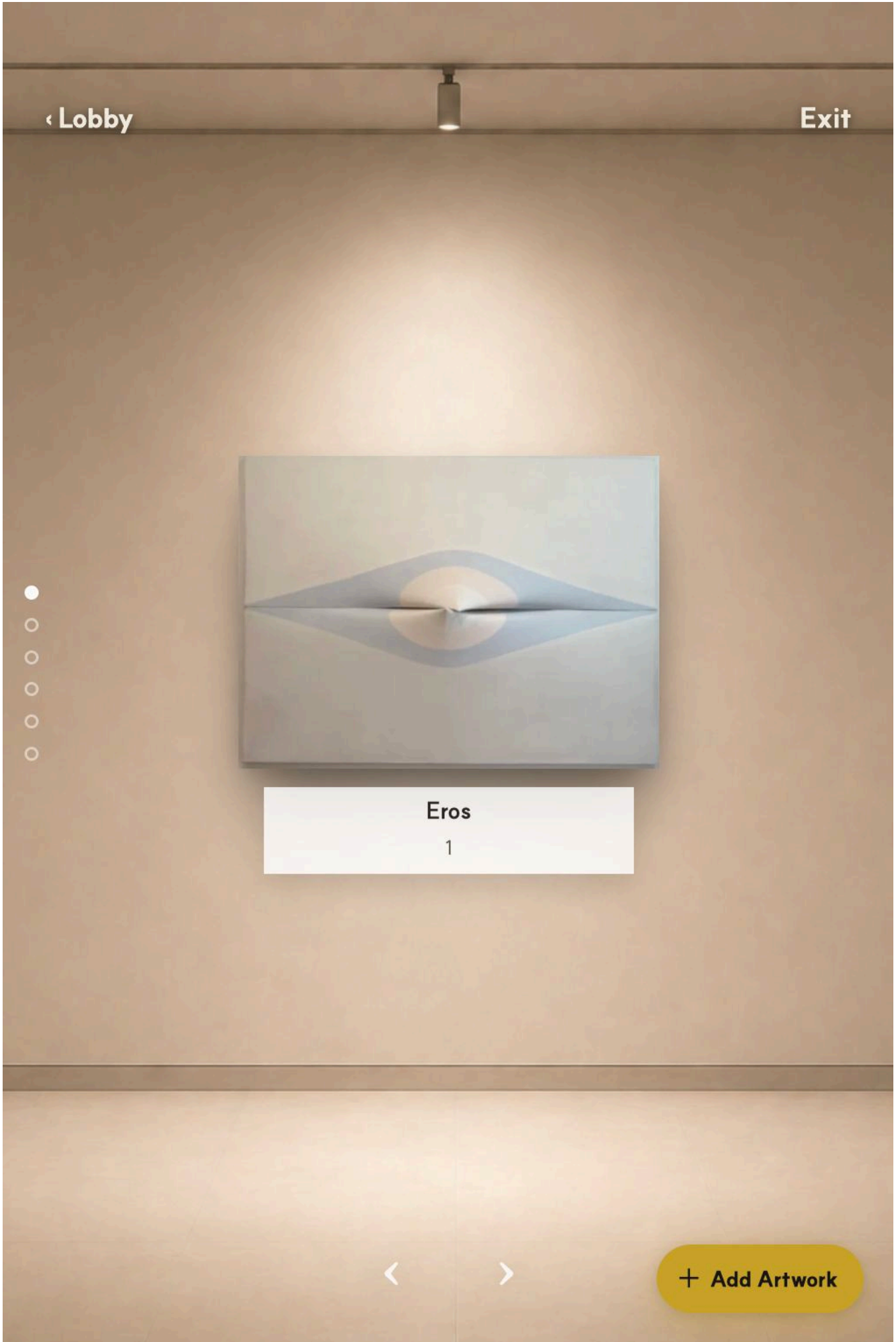
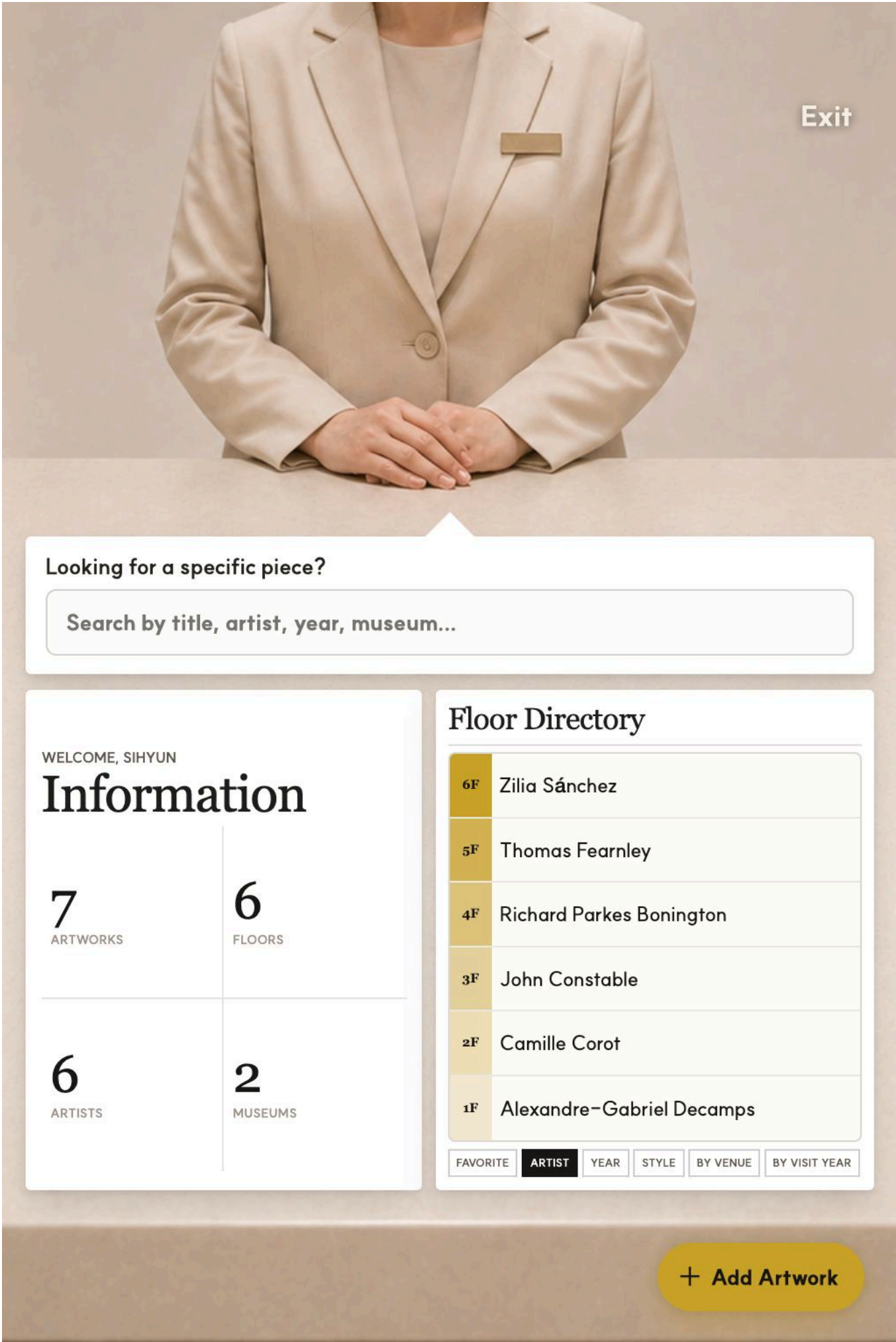
Final Image

Web Page



Final Image

Media Query



Source Code

Canvas API — Crop & Rotation

JS Code

```
function cropLoadedImage(image, item) {  
  const deg = getNum(item, "cropRotate", 0);  
  const rad = deg * Math.PI / 180;  
  const selX = getNum(item, "selX", 0) / 100;  
  const selY = getNum(item, "selY", 0) / 100;  
  const selW = getNum(item, "selW", 100) / 100;  
  const selH = getNum(item, "selH", 100) / 100;  
  
  const FRAME = 800;  
  const r = image.naturalWidth / image.naturalHeight;  
  const dw = r >= 1 ? FRAME : FRAME * r;  
  const dh = r >= 1 ? FRAME / r : FRAME;  
  
  const fc = document.createElement("canvas");  
  fc.width = FRAME;  
  fc.height = FRAME;  
  const ctx = fc.getContext("2d");  
  ctx.save();  
  ctx.translate(FRAME / 2, FRAME / 2);  
  ctx.rotate(rad);  
  ctx.drawImage(image, -dw / 2, -dh / 2, dw, dh);  
  ctx.restore();  
  
  const cx = selX * FRAME, cy = selY * FRAME;  
  const cw = Math.max(1, selW * FRAME), ch = Math.max(1, selH * FRAME);  
  
  const MAX = 1200;  
  const or = cw / ch;  
  const ow = or >= 1  
    ? Math.min(Math.round(cw / FRAME * image.naturalWidth), MAX)  
    : Math.round(Math.min(Math.round(ch / FRAME * image.naturalHeight), MAX) * or);  
  const oh = Math.round(ow / or);  
  
  const oc = document.createElement("canvas");  
  oc.width = Math.max(ow, 1);  
  oc.height = Math.max(oh, 1);  
  oc.getContext("2d").drawImage(fc, cx, cy, cw, ch, 0, 0, ow, oh);  
  return oc.toDataURL("image/webp", 0.86);  
}
```

Prepare Crop/Rotate values

Fit the original image to an 800×800 reference frame.

Creating a Rotated Image

Calculate Crop Area

Output Image Size Calculation

Create Final Cropped Image

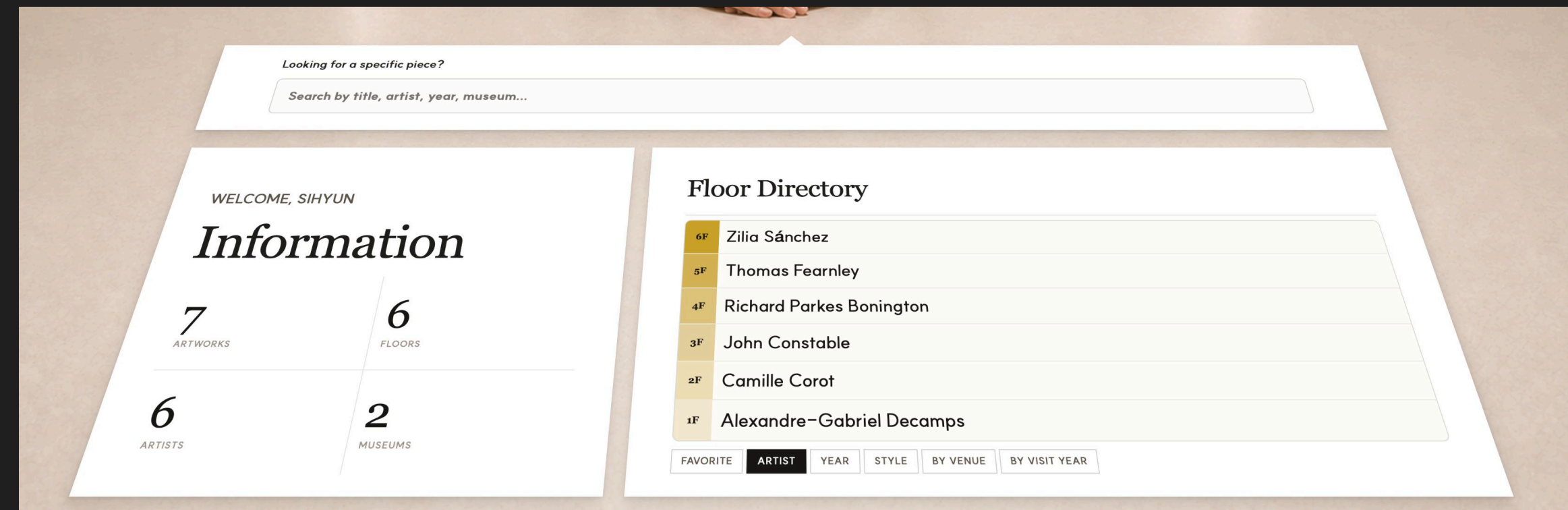
```
const ctx = canvas.getContext("2d");
```

```
ctx.save();  
ctx.translate(FRAME / 2, FRAME / 2);  
ctx.rotate(rad);  
ctx.drawImage(image, -dw / 2, -dh / 2, dw, dh);  
ctx.restore();
```


Additional Issue

Perspective

```
.lobby-desk {  
  position: absolute;  
  bottom: 12vh;  
  left: 0;  
  right: 0;  
  height: 58vh;  
  padding: 0 clamp(2rem, 5%, 7rem);  
  display: grid;  
  grid-template-columns: 3fr 5fr;  
  grid-template-rows: auto 1fr;  
  column-gap: clamp(0.8rem, 1.5%, 2rem);  
  row-gap: 0;  
  perspective: 1500px;  
  perspective-origin: 50% 0%;  
}
```



Next Steps

If I had more time...

AI-powered artwork recognition

Currently, artwork details (title, artist, year, museum) must be typed in manually after uploading a photo.

With more time and tools, I would integrate an AI vision API so that when a user photographs an artwork, the app automatically detects and fills in the metadata — recognising the piece from its visual appearance.

Gallery sharing

Export or share a curated selection of artworks as a public link — letting visitors browse a read-only version of your collection.

Visit log & map

Track which museums have been visited and visualise them on an interactive world map, grouped by city or country.

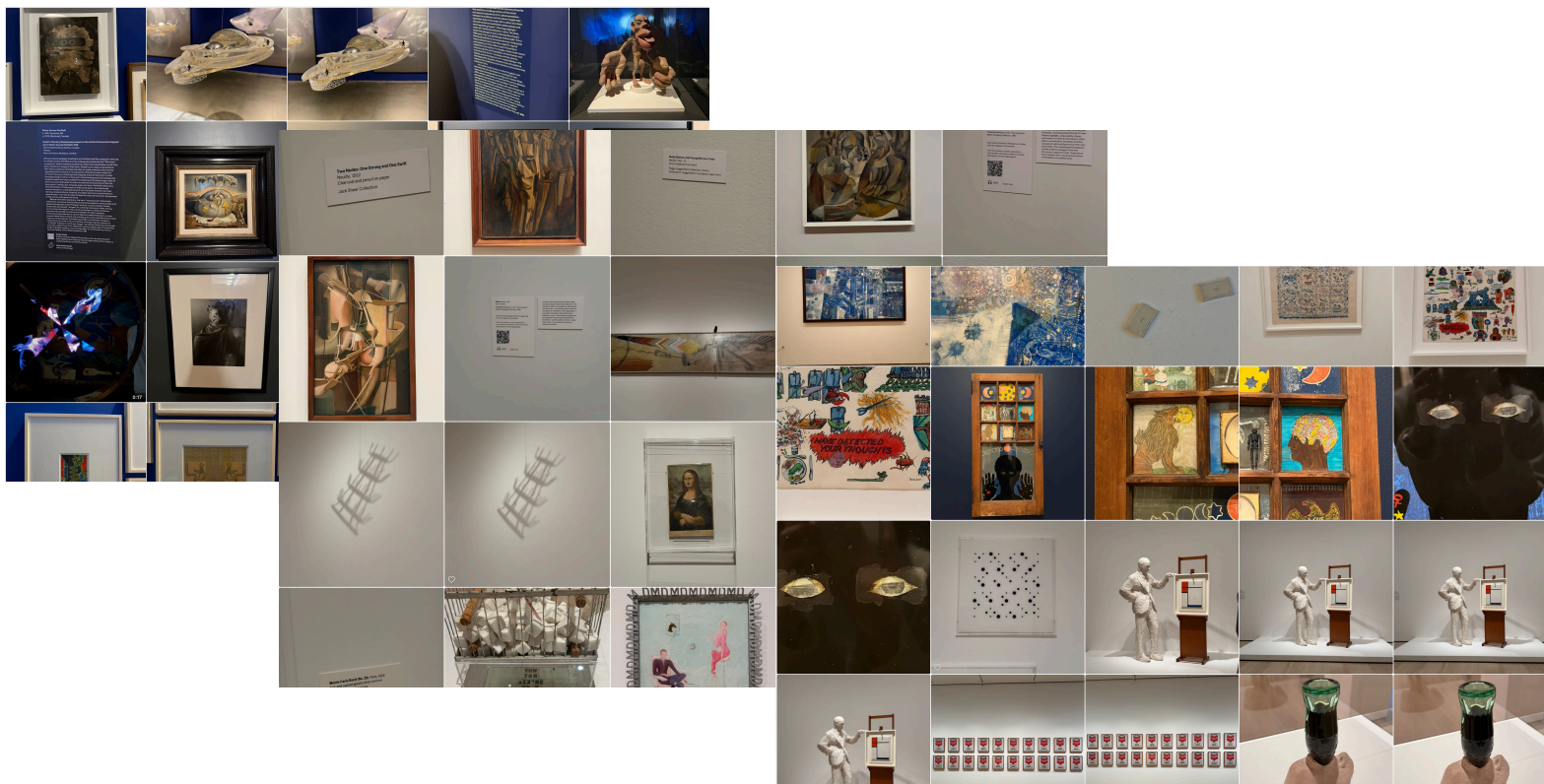
Intro to Web Development

Personal Art Collection Website

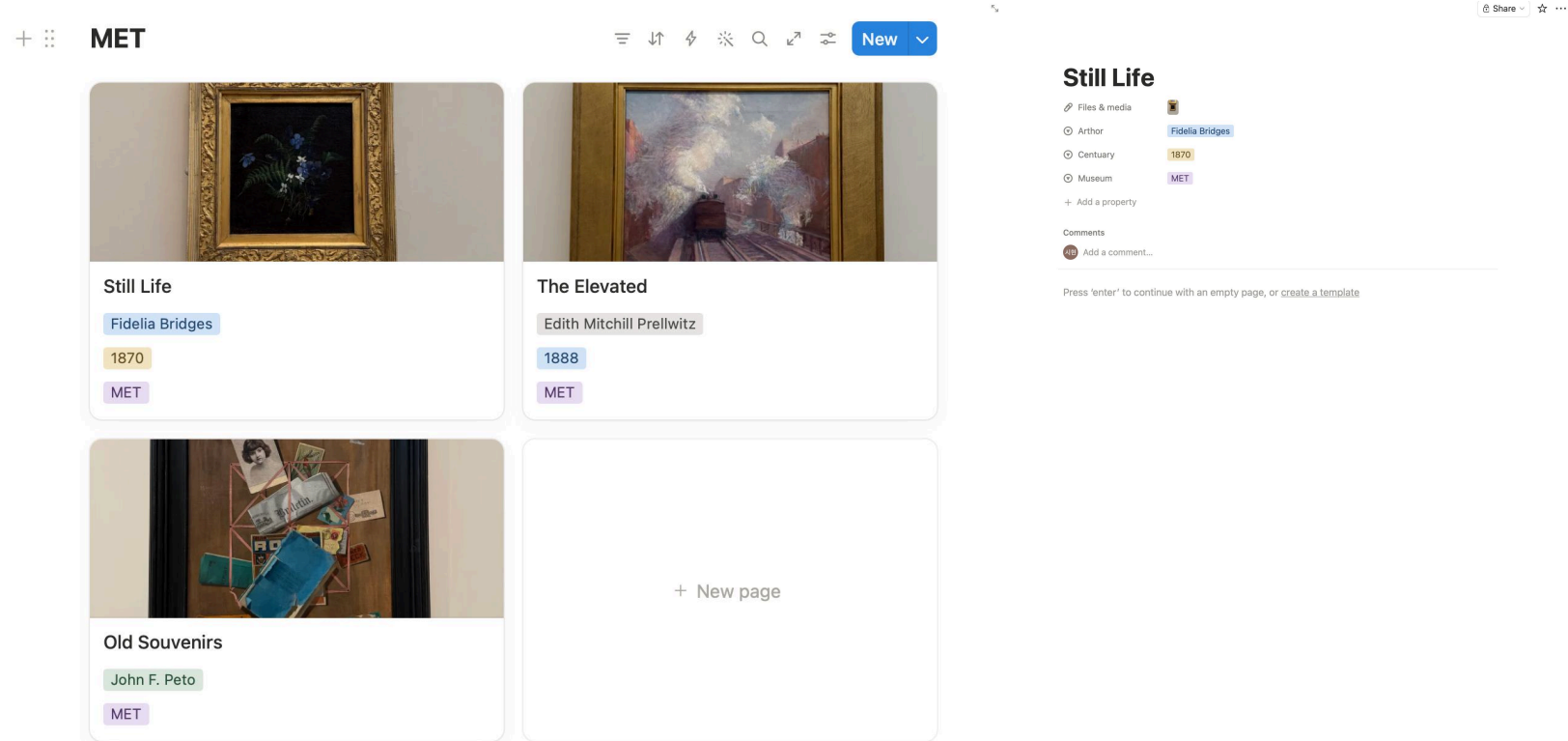
A personal digital gallery for art encountered during museum visits

Problem

My album after a visit to the museum



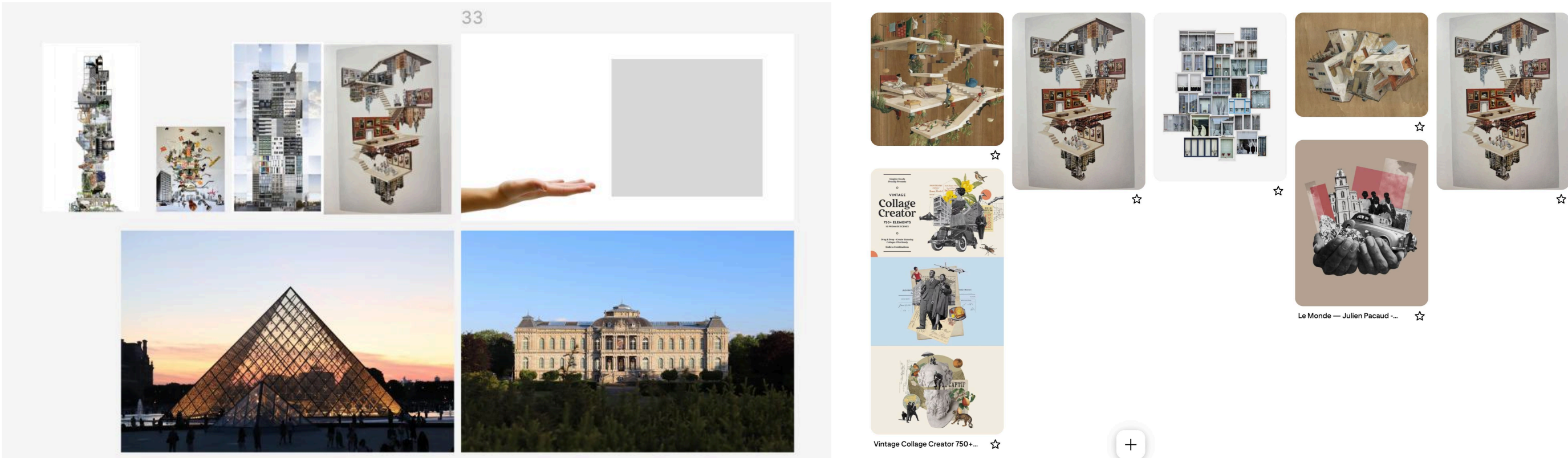
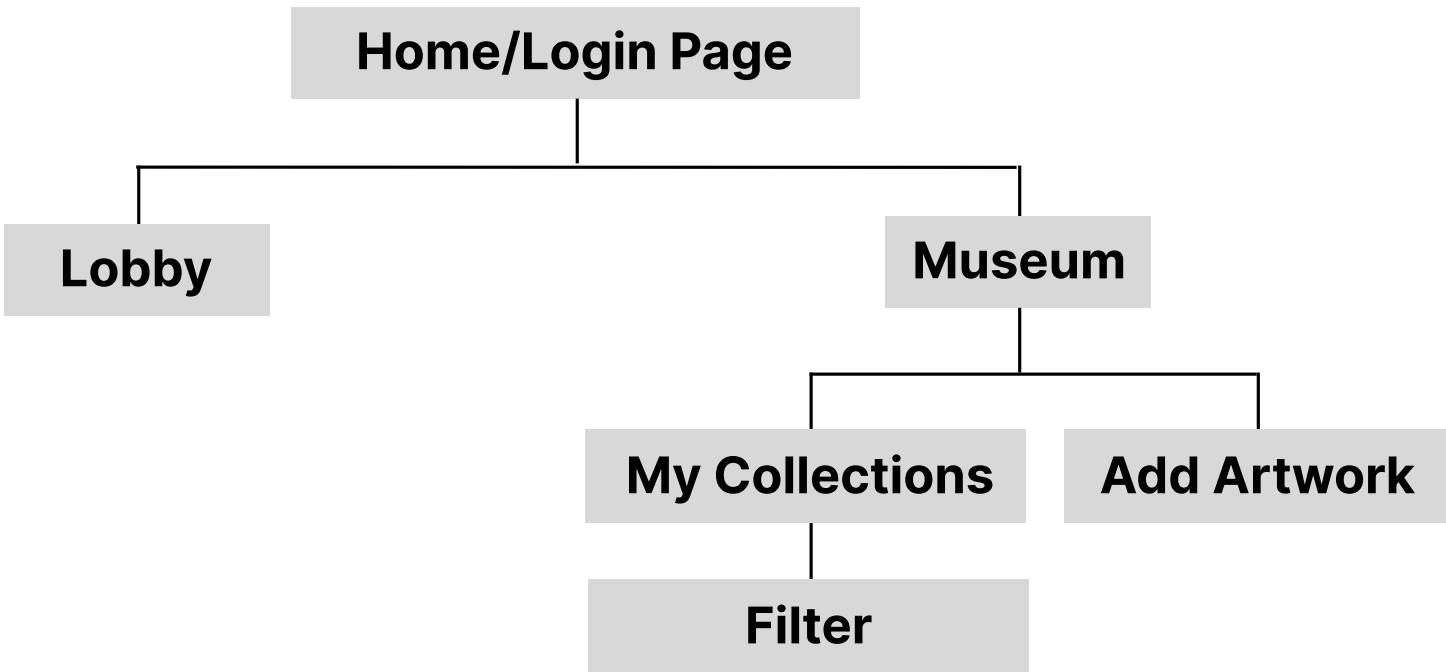
Tried doing it with Notion, but it didn't work out well



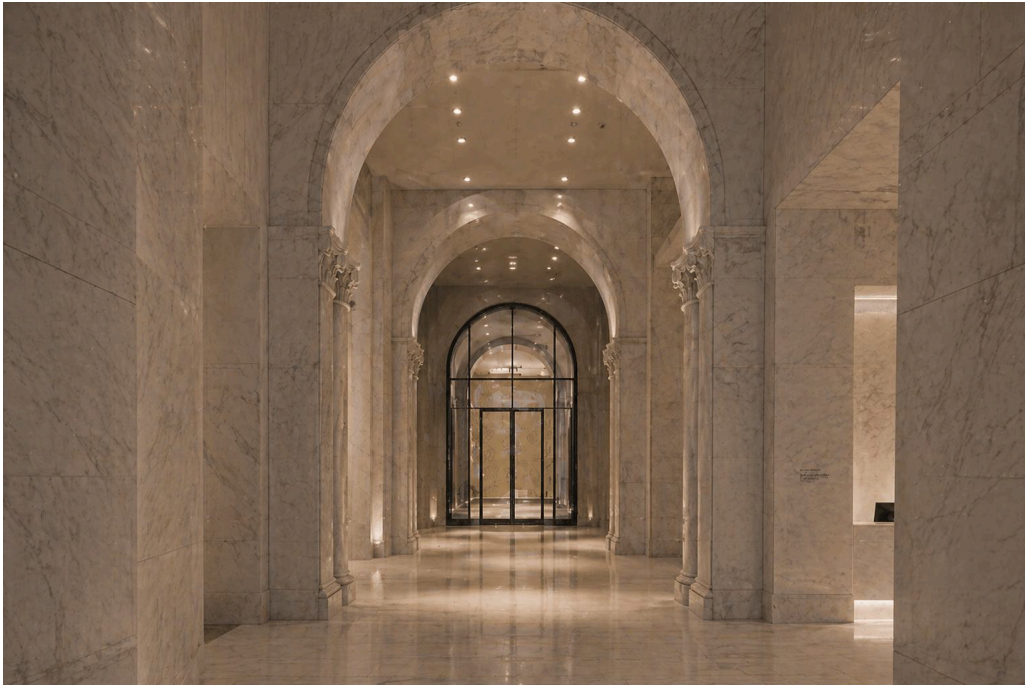
Solution



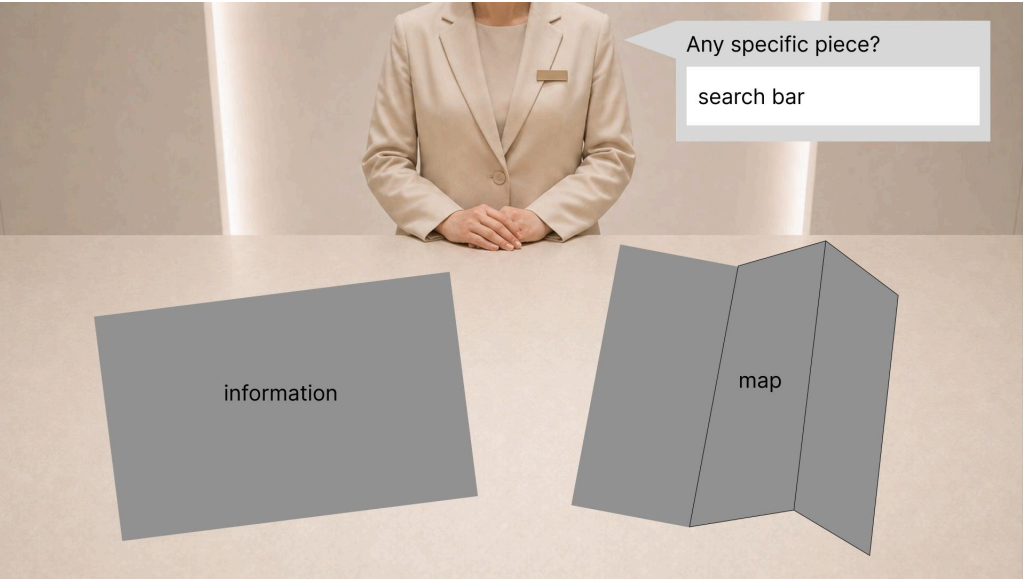
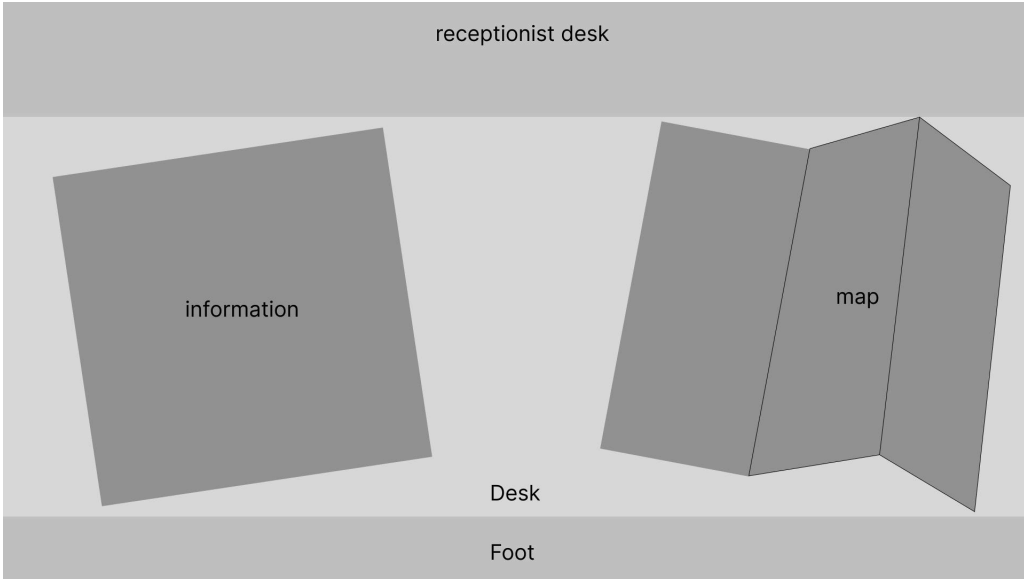
IA & Design Reference



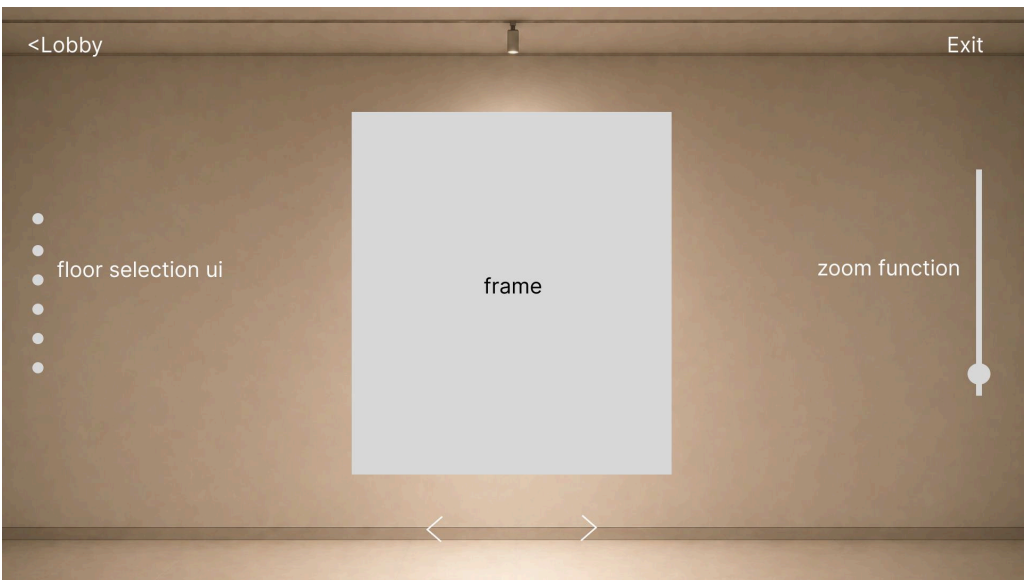
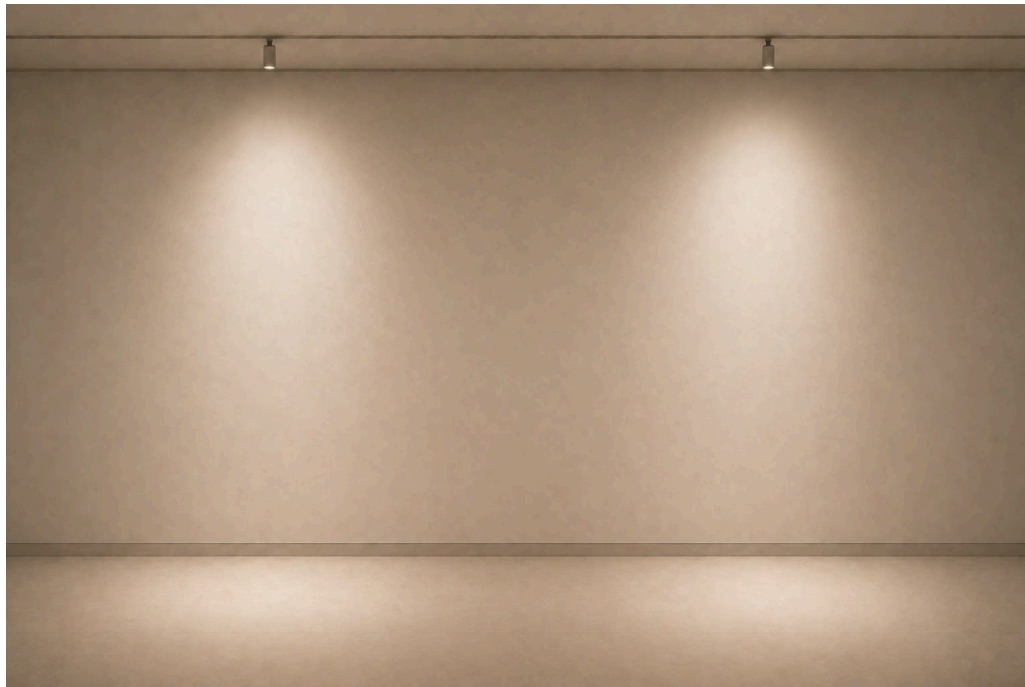
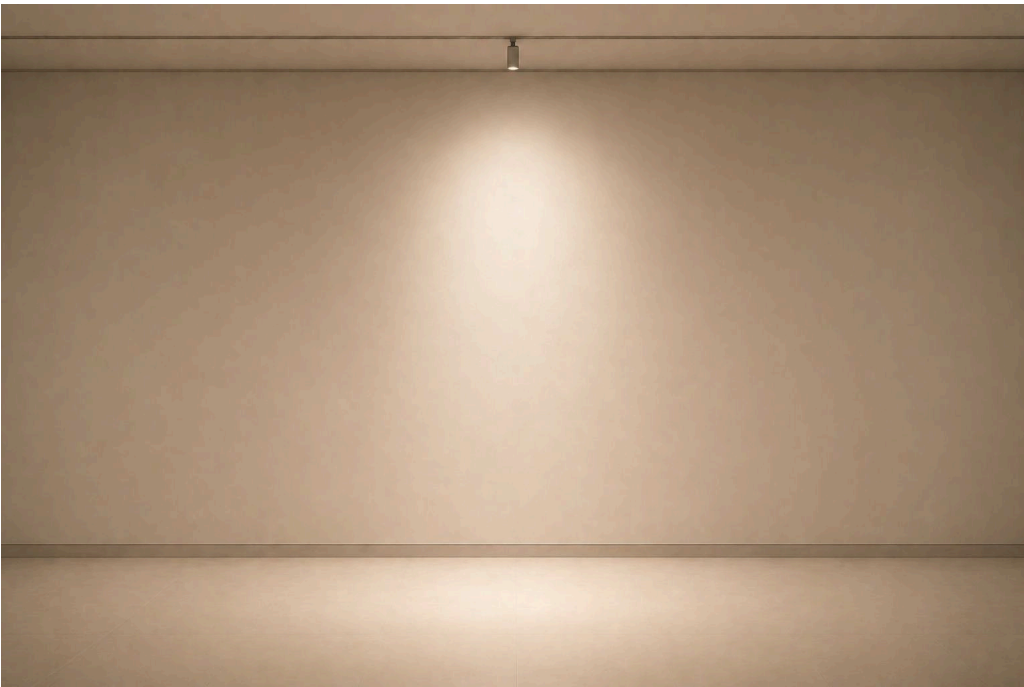
Entrance



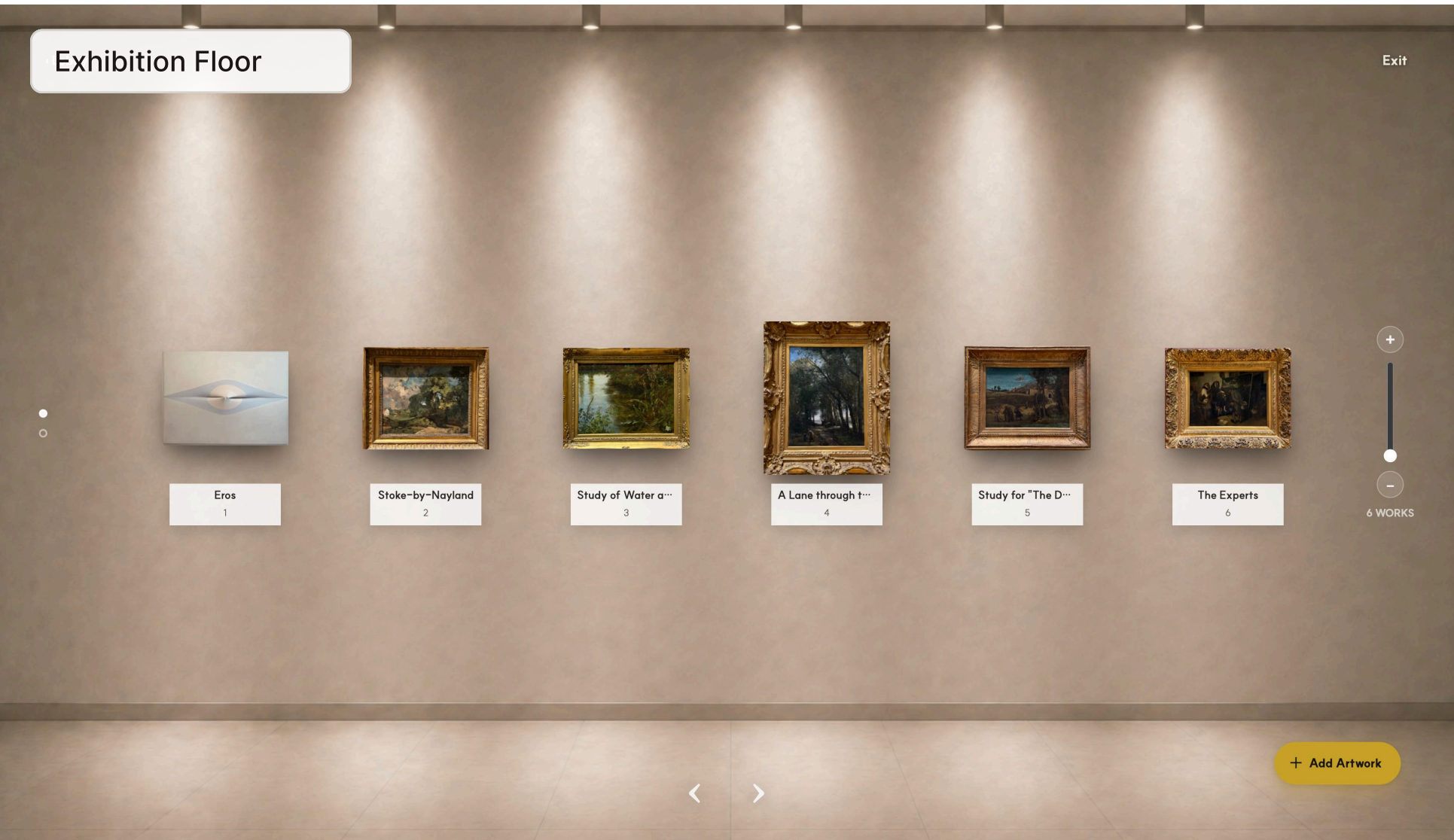
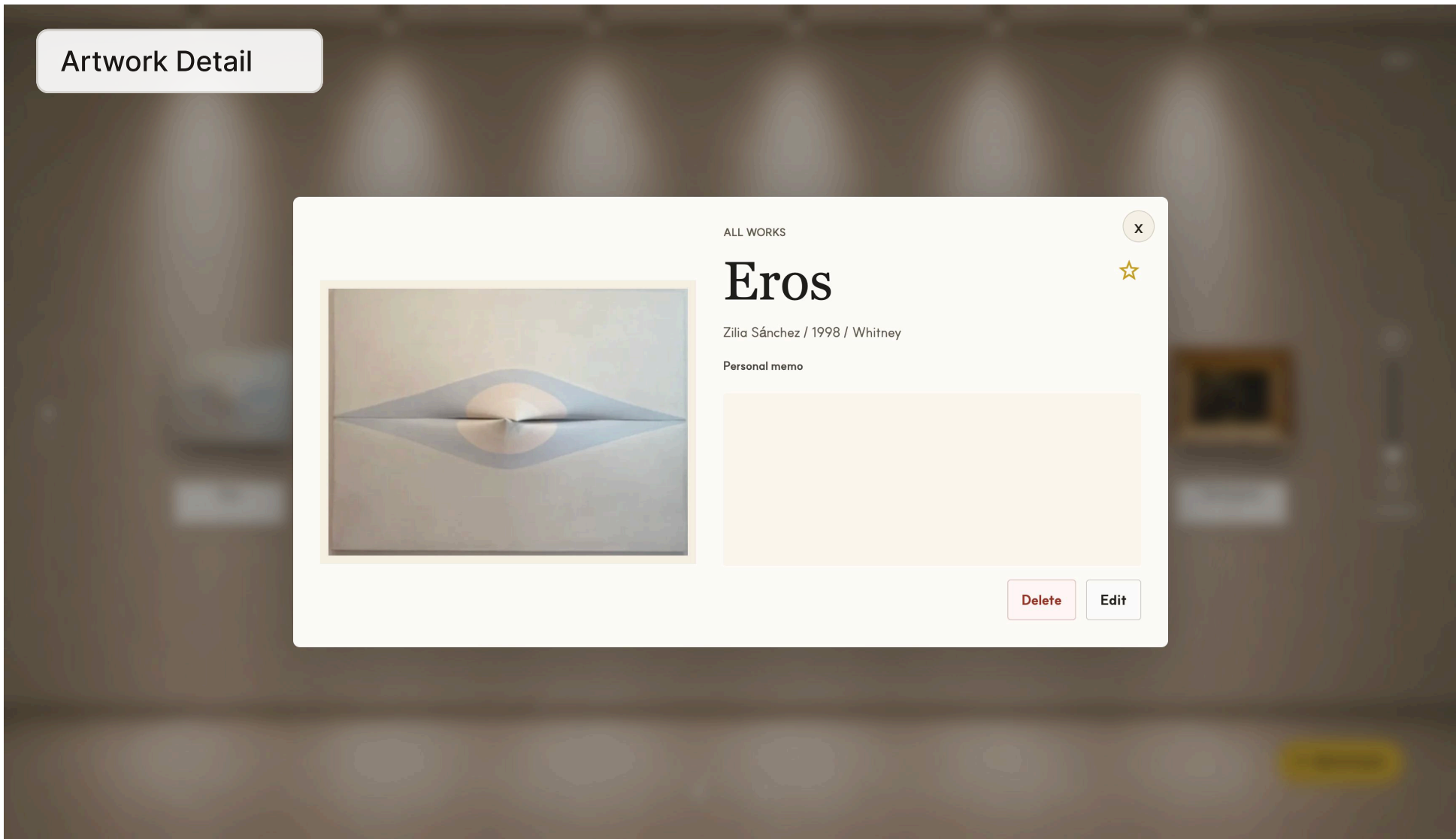
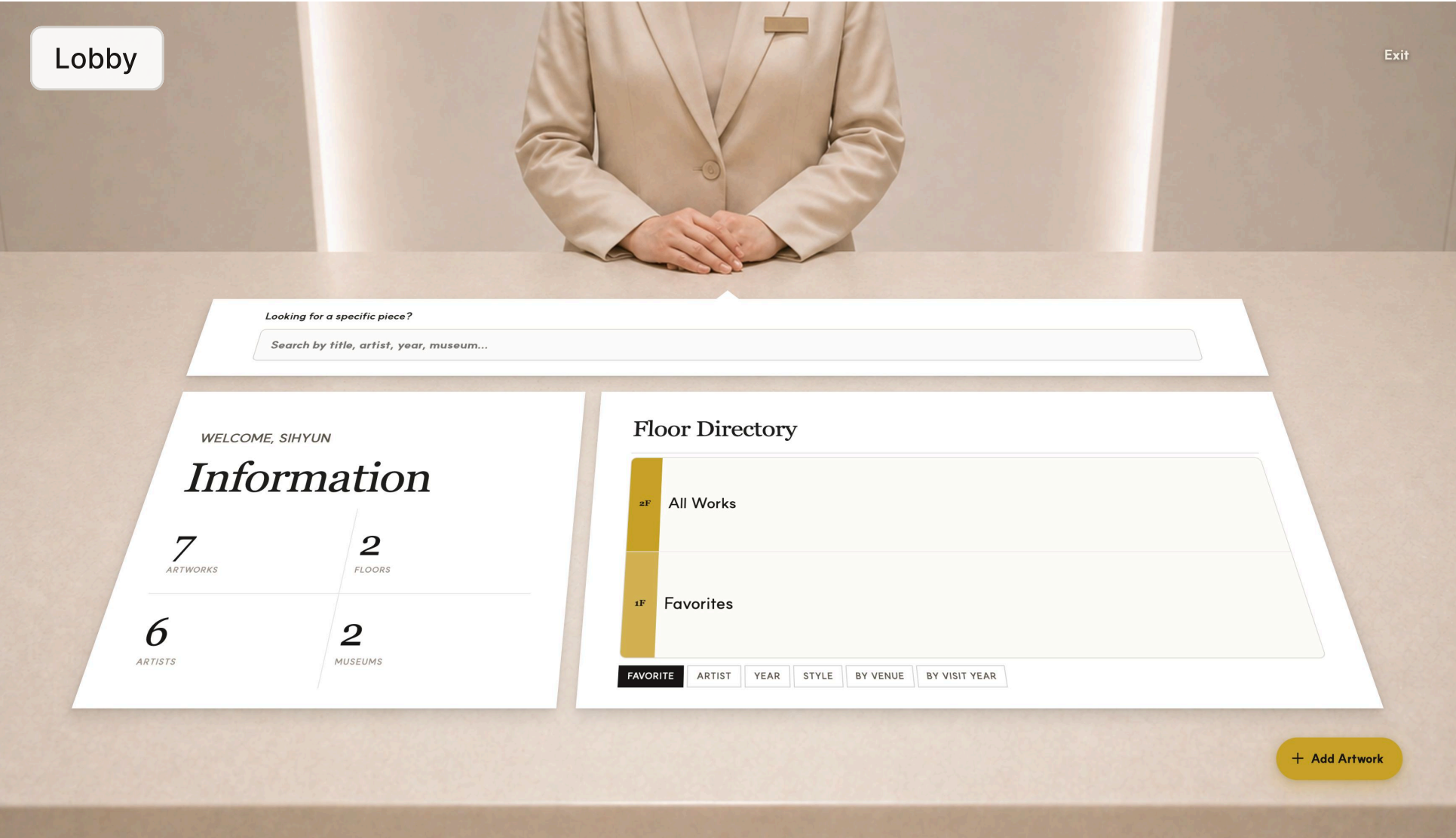
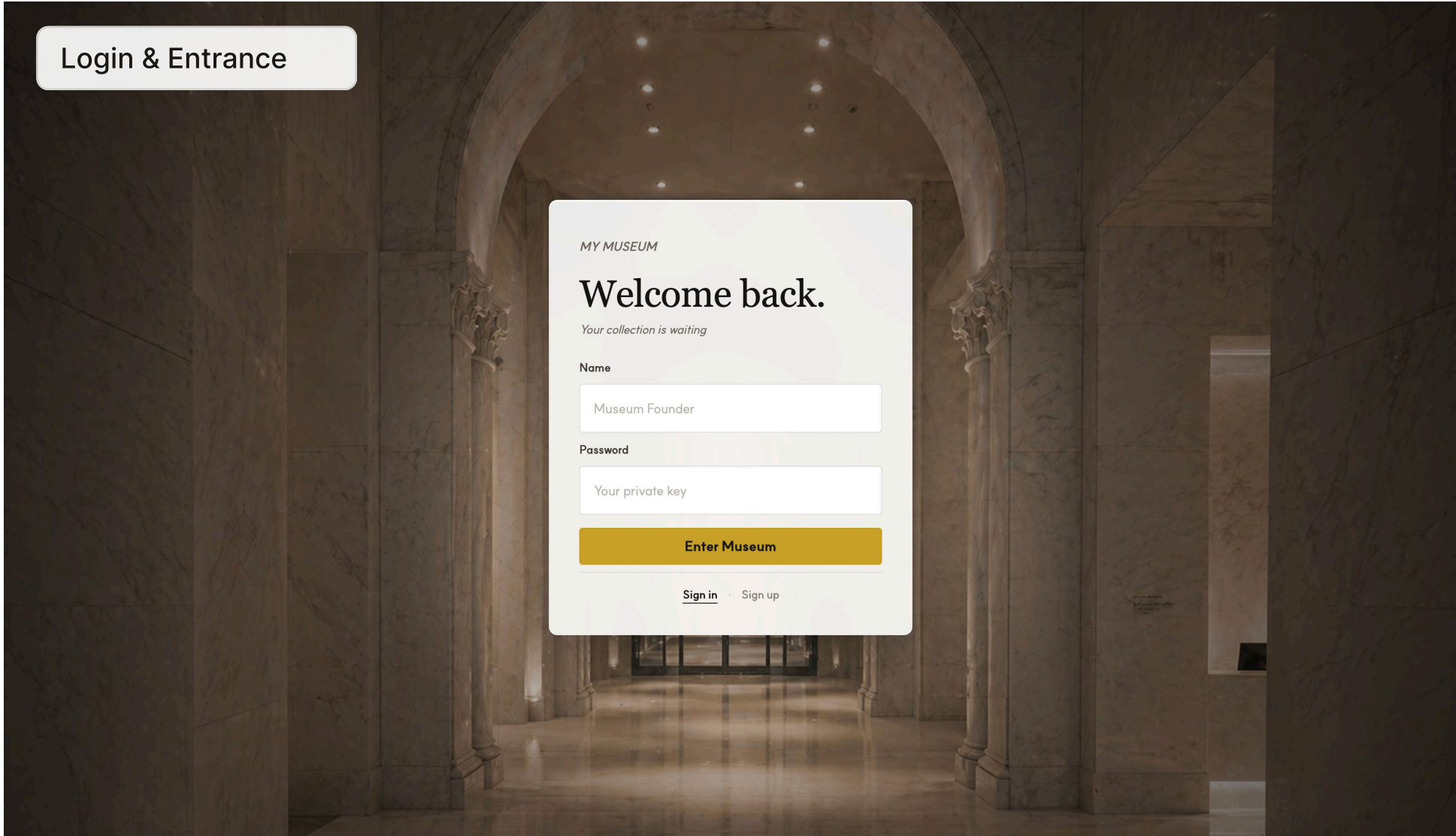
Lobby



Inside



Website



Source Code

Canvas API — Crop & Rotation

The Challenge

When users upload artwork photos, they need precise control over framing and orientation.

A custom crop tool was built using the Canvas API — an 800×800 virtual canvas is created, the image is drawn with `ctx.rotate()` applied, then only the selected rectangle is extracted.

For the spotlight overlay, instead of adding a separate dark element, a single thick box-shadow on the selection box itself creates the darkened surround effect.

Code

```
// Rotate image on a virtual 800×800 canvas
ctx.save();
ctx.translate(cx, cy);
ctx.rotate((deg * Math.PI) / 180);
ctx.drawImage(
  img,
  -img.naturalWidth / 2,
  -img.naturalHeight / 2,
  img.naturalWidth,
  img.naturalHeight
);
ctx.restore();

// Extract only the selected region
const out = document.createElement("canvas");
out.width = selW;
out.height = selH;
outCtx.drawImage(canvas, selX, selY, selW, selH, 0, 0, selW, selH);

// Spotlight: no extra overlay element needed
/* CSS on the selection box */
box-shadow: 0 0 0 9999px rgba(0, 0, 0, 0.48);
```


Source Code

CSS Subgrid — Caption Alignment

The Problem

Each artwork has a different aspect ratio. Captions placed directly below each image ended up at different vertical positions — making the gallery wall feel uneven.

Result

All captions now sit at the same absolute Y position on screen, regardless of each painting's proportions.

Why it matters

CSS Subgrid is a modern feature that lets child elements participate directly in a grandparent grid's tracks — no JavaScript, no absolute positioning, no hacky equal-height tricks.

Code

```
/* 2-row grid: images above, captions below */
.gallery-wall {
  grid-template-rows: 1fr auto;
  column-gap: clamp(0.8rem, 2vw, 2rem);
  row-gap: 0.65rem;
}

/* Each card spans both rows via subgrid */
.art-card {
  display: grid;
  grid-template-rows: subgrid;
  grid-row: span 2;
}

/* art-button inherits the same row tracks */
.art-button {
  display: grid;
  grid-template-rows: subgrid;
  grid-row: span 2;
}

/* Image centers in row 1 */
.art-image { align-self: center; }

/* Caption anchors to top of row 2 */
.art-caption { align-self: start; }
```


Issues Encountered

01

Rotation slider not working

initCropFrame() queried the DOM for the slider before the preview and input elements were appended to the page — so `querySelector` returned null and the event listener was never attached.

Fix: reorder so `append()` runs first, then `initCropFrame()` is called.

02

Slider still broken after the reorder fix

The `.artwork-form` input CSS rule was also targeting `<input type="range">`, overriding the browser's native range thumb with form-field styles and breaking interaction.

Fix: add `:not([type="range"])` to the selector so the rule only applies to text inputs.

03

Images always showing the top portion instead of centre

`object-fit: cover` defaults to clipping from the top when the container has no explicit aspect ratio set. For artworks with natural proportions, the container height was unpredictable.

Fix: after the image loads, read `img.naturalWidth / naturalHeight` and set it as the container's inline `style.aspectRatio`.

What I Learned

01	Canvas API	Rotating and compositing images by translating the context origin, applying rotation, then drawing — rather than transforming the DOM element itself.
02	Pointer Events API	Using <code>setPointerCapture()</code> so drag interactions keep firing even after the cursor leaves the element bounds — essential for the crop handle UX.
03	CSS Subgrid	Letting child elements participate in a grandparent grid's row tracks, making cross-card caption alignment possible without any JavaScript.
04	localStorage Architecture	Structuring per-user data with hashed keys so multiple accounts can share one browser without any data leaking between sessions.
05	CSS perspective & rotateX	Setting perspective on a parent container so all children share a single vanishing point — creating the 3D tilted-desk effect in the lobby.

Next Steps

If I had more time...

AI-powered artwork recognition

Currently, artwork details (title, artist, year, museum) must be typed in manually after uploading a photo.

With more time and tools, I would integrate an AI vision API so that when a user photographs an artwork, the app automatically detects and fills in the metadata — recognising the piece from its visual appearance.

If a wall label or caption is visible in the photo, OCR would extract the text directly, so no typing is needed at all.

Image Recognition

OCR

Auto-fill Metadata

Gallery sharing

Export or share a curated selection of artworks as a public link — letting visitors browse a read-only version of your collection.

Visit log & map

Track which museums have been visited and visualise them on an interactive world map, grouped by city or country.